Algorithm Analysis and Design

1. Dynamic Programming Fundamentals

Dynamic programming is an optimization technique that solves complex problems by breaking them down into simpler subproblems. It is applicable when a problem exhibits optimal substructure and overlapping subproblems.

Key principles:
- Identify recursive structure of the problem
- Store solutions to subproblems to avoid redundant computation
- Build solutions bottom-up or use memoization top-down

Classic examples include the Fibonacci sequence, longest common subsequence, and the knapsack problem. Dynamic programming can reduce exponential time complexity to polynomial time in many cases.

The Bellman equation provides a mathematical foundation for dynamic programming, expressing the value of a decision problem at a certain point in time in terms of the payoff from initial choices and the value of the remaining problem.

2. Graph Traversal Algorithms

Graph traversal explores all vertices and edges of a graph systematically. The two fundamental approaches are breadth-first search (BFS) and depth-first search (DFS).

BFS explores nodes level by level using a queue data structure. It finds shortest paths in unweighted graphs and has applications in network routing and social network analysis.

DFS explores as far as possible along each branch using a stack (or recursion). It is useful for topological sorting, cycle detection, and solving maze problems. The time complexity for both algorithms is O(V + E) where V is vertices and E is edges.

Advanced graph algorithms build upon these traversal methods. Dijkstra algorithm extends BFS to handle weighted graphs, while strongly connected components can be found using modified DFS. Understanding these fundamental traversal patterns is essential for mastering more complex graph algorithms.